BIT 2118: SCIENTIFIC COMPUTING

Group Work

To be completed by Week 10. Group Project task to be done in Week 11-13.

Instructions

A. All Groups to do all the Tasks.
B. Identify the input values and the mathematical operation performed.
C. Change at least one input/parameter and explain how the result changes.
D. Explain the meaning of the output in words and interpret the graph where necessary.
E. Modify the program to solve a different but related real-world case. Document the new assumptions, input data, output and interpretation in the individual GitHub repository.

F. The discussions will be conducted as group work; however, the GitHub submission is individual. Each student should maintain and submit their own GitHub repository. Group repositories should not be used.

G. Upload of the github links and documentation will be done via VLMS.

Tasks	Mathematical concept	Example application
Prerequisite	Data Science Essentials with Python (Self-paced)	Complete before undertaking the Scientific Computing presentation tasks.
2	NumPy Arrays and Numerical Computation	Sensor readings from IoT devices
3	Mathematical Functions and Vectorization	Large datasets, simulations, signal processing, image processing, numerical models and machine learning where millions of arithmetic operations are repeated.
4	Floating-Point Arithmetic and Numerical Errors	Scientific simulations, banking calculations, GPS/navigation, engineering design, iterative algorithms and any system where accumulated numerical error can affect a result.
5	Matrix Algebra and Matrix Operations	Computer graphics transformations, machine learning, cryptography, network models, robotics, electrical circuits and solving systems of equations.
6	Solution of Systems of Linear Equations	Electrical circuit analysis, resource allocation, traffic-flow models, balancing chemical equations, computer graphics

		and estimating unknown parameters.
7	Gaussian Elimination and LU Decomposition	Engineering simulations, repeated linear-system solutions, finite-element methods, economic models and scientific models with many equations.
8	Iterative Methods for Linear Systems	Large sparse systems from heat transfer, electrical networks, structural models, image reconstruction and scientific simulations where direct matrix factorization may be expensive.
9	Eigenvalues and Eigenvectors	Google PageRank concepts, vibration analysis, stability of dynamical systems, principal component analysis, image compression, quantum models and network centrality.
10	Root Finding: Bisection and Fixed-Point Methods	Finding break-even points, solving engineering equations, computing interest rates, calibration, physical equilibrium and threshold values.
11	Root Finding: Newton-Raphson and Secant Methods	Circuit equations, optimization subproblems, financial rate calculations, nonlinear models, calibration and solving physical equations.
12	Polynomial and Nonlinear Equation Solving	Kinematics, chemical equilibrium, geometric intersection, control systems, calibration and finding operating points in nonlinear models.
13	Interpolation	Sensor calibration, missing measurement estimation, animation, geographic data, lookup tables, engineering tables and resampling signals.
14	Curve Fitting and Approximation	Trend estimation, calibration, performance modelling, growth curves, sensor models, demand forecasting and experimental science.
15	Numerical Differentiation	Estimating speed from position data, acceleration from velocity, gradients in optimization, rates of change in sensors and sensitivity analysis.

16	Numerical Integration	Energy consumption, distance from velocity data, probability calculations, signal energy, total flow, area and accumulated quantities.
17	Ordinary Differential Equations – Euler Method	Population growth, cooling, charging capacitors, motion, epidemic models and other systems that evolve over time.
18	Numerical Solution of ODEs Using SciPy	Control systems, epidemiology, robotics, chemical reactions, electrical circuits, mechanical systems and dynamic simulations.
19	Numerical Optimization	Cloud resource allocation, network routing, machine-learning training, scheduling, engineering design, cost reduction and parameter tuning.
20	Random Numbers and Monte Carlo Methods	Risk analysis, reliability, queue simulation, finance, uncertainty analysis, computer graphics, physical simulations and estimating difficult integrals.

Python Libraries and Their Uses

Library / Module	Purpose	Typical Import	Common Functions
NumPy	Stores numerical data in arrays and performs fast vectorized calculations, matrix operations and linear algebra.	import numpy as np	np.array(), np.mean(), np.linalg.solve(), np.linalg.eig()
SciPy	Provides scientific algorithms for equation solving, interpolation, curve fitting, integration, ODEs and optimization.	from scipy...	root_scalar(), root(), curve_fit(), simpson(), solve_ivp(), minimize()
Matplotlib	Creates graphs for interpretation of convergence, fitted curves, ODE	import matplotlib.pyplot as plt	plt.plot(), plt.scatter(), plt.xlabel(), plt.legend(), plt.show()

	behaviour, interpolation and simulation results.		
timeit	Measures execution time to compare a Python loop with vectorized NumPy code.	import timeit	timeit.timeit(function, number=1)

SciPy Submodules Used

Submodule	What It Is Used For	Functions Used	Task(s)
scipy.linalg	LU decomposition and repeated solutions of linear systems	lu_factor(), lu_solve()	Task 7
scipy.optimize	Root finding, nonlinear systems, curve fitting and optimization	root_scalar(), root(), curve_fit(), minimize()	Task 10-12, 14, 19
scipy.interpolate	Estimation of values between measured data points	interp1d(), CubicSpline()	Task 13
scipy.integrate	Numerical integration and numerical solution of differential equations	simpson(), solve_ivp()	Task 16, 18

A. NumPy Arrays and Numerical Computation

1. Mathematical Concept

An array is an organized collection of numbers. A vector is a one-dimensional array such as v = [v1, v2, ..., vn], while a matrix is a rectangular array A = [aij]. Scientific computing often applies one operation to every element. For Example, if x = [1,2,3], then x^2 = [1,4,9]. NumPy performs these operations directly without writing a loop for each value.

2. Example

If temperatures are [20, 22, 24, 26] °C, their mean is (20+22+24+26)/4 = 23 °C. With NumPy, np.mean(temperatures) performs this calculation immediately.

3.. Example Real World Applications

1. Sensor readings from IoT devices
2. image pixels

3. network traffic measurements
4. scientific measurements
5. machine-learning feature matrices
6. financial time-series.

4. Real-world problem and solution approach

Problem: A weather station records 24 hourly temperatures. Store them in an array, calculate the daily mean, minimum, maximum and convert all values from Celsius to Fahrenheit.

Solution approach: Represent the readings as one NumPy array. Use mean(), min() and max(), then apply F = 1.8C + 32 to the entire array at once.

5. Solution Code

Program file: code/Task_02_numpy_arrays.py

```
import numpy as np

# 24 hourly temperature readings in degrees Celsius
temperatures_c = np.array([
    18.5, 18.0, 17.8, 17.5, 17.9, 19.0,
    21.5, 23.8, 25.6, 27.1, 28.3, 29.0,
    29.4, 29.1, 28.5, 27.6, 26.2, 24.8,
    23.4, 22.1, 21.0, 20.2, 19.6, 19.0
])

mean_temp = np.mean(temperatures_c)
min_temp = np.min(temperatures_c)
max_temp = np.max(temperatures_c)

# Vectorized Celsius-to-Fahrenheit conversion
temperatures_f = 1.8 * temperatures_c + 32

print(f"Daily mean temperature: {mean_temp:.2f} C")
print(f"Minimum temperature: {min_temp:.2f} C")
print(f"Maximum temperature: {max_temp:.2f} C")
print("First six Fahrenheit readings:", temperatures_f[:6])
```

B.Mathematical Functions and Vectorization

1. Mathematical Concept

Vectorization means applying a mathematical operation to many values at the same time. If $y_i = \sin^2(x_i) + \cos^2(x_i)$, the identity gives $y_i = 1$ for every x_i. Instead of looping through x1, x2, ..., xn, NumPy evaluates the whole vector in one expression.

2. Example

For x = [0, π/2, π], compute sin(x). The result is approximately [0, 1, 0]. The same operation works whether x contains 3 values or one million values.

3. Example Real World Applications

Large datasets, simulations, signal processing, image processing, numerical models and machine learning where millions of arithmetic operations are repeated.

4. Real-world problem and solution approach

Problem: A network-monitoring system has one million latency measurements and needs to convert each from milliseconds to seconds and square the values for an error calculation.

Solution approach: Use vectorized NumPy expressions such as seconds = latency_ms/1000 and squared = seconds**2. This is simpler and usually much faster than Python loops.

5. Solution Code

Program file: code/Task_03_vectorization.py

```
import numpy as np
import timeit

# Simulated network latency readings in milliseconds
rng = np.random.default_rng(42)
latency_ms = rng.uniform(5, 250, 1_000_000)

def loop_version():
    squared = []
    for value in latency_ms:
        seconds = value / 1000.0
        squared.append(seconds ** 2)
    return squared

def vectorized_version():
    seconds = latency_ms / 1000.0
    return seconds ** 2

loop_time = timeit.timeit(loop_version, number=1)
vector_time = timeit.timeit(vectorized_version, number=3) / 3

result = vectorized_version()

print("First five squared latency values:", result[:5])
print(f"Loop time: {loop_time:.4f} s")
print(f"Vectorized time: {vector_time:.4f} s")
print(f"Approximate speed-up: {loop_time/vector_time:.1f}x")
```

C. Floating-Point Arithmetic and Numerical Errors

1. Mathematical Concept

Computers store most real numbers with a finite number of binary digits, so some values are approximations. Absolute error is |true value - approximate value|. Relative error is absolute error / |true

value|. Truncation error comes from replacing an exact mathematical process with an approximation; round-off error comes from finite computer precision.

2. Example

If √2 = 1.41421356 but we use 1.414, the absolute error is about 0.00021356. Relative error is about 0.000151 or 0.0151%.

3. Example Real World Applications

Scientific simulations, banking calculations, GPS/navigation, engineering design, iterative algorithms and any system where accumulated numerical error can affect a result.

4. Real-world problem and solution approach

Problem: A program compares 0.1 + 0.2 with 0.3 and unexpectedly reports False.

Solution approach: This happens because the binary floating-point representation is approximate. Compare using np.isclose() or a suitable tolerance instead of exact equality.

5. Solution Code

Program file: code/Task_04_numerical_errors.py

```
import numpy as np

# Floating-point comparison
calculated = 0.1 + 0.2
expected = 0.3

print("0.1 + 0.2 =", calculated)
print("Exact comparison:", calculated == expected)
print("Tolerance-based comparison:", np.isclose(calculated, expected))

# Absolute and relative error
true_value = np.sqrt(2)
approx_value = 1.414

absolute_error = abs(true_value - approx_value)
relative_error = absolute_error / abs(true_value)

print(f"Absolute error: {absolute_error:.8f}")
print(f"Relative error: {relative_error:.8%}")

# Numerically unstable vs stable formulation
for x in [1e-4, 1e-8, 1e-12]:
    direct = (np.sqrt(1 + x) - 1) / x
    stable = 1 / (np.sqrt(1 + x) + 1)
    print(f"x={x:.0e}: direct={direct:.12f}, stable={stable:.12f}")
```

D. Matrix Algebra and Matrix Operations

1. Mathematical Concept

A matrix organizes values in rows and columns. Matrix addition combines corresponding elements. Matrix multiplication combines rows of the first matrix with columns of the second: $c_{i□} = Σ\ a_{i□}b_{□□}$. The determinant helps indicate whether a square matrix is invertible. The inverse A^{-1} satisfies $AA^{-1} = I$.

2. Example

For A=[[1,2],[3,4]] and x=[5,6], Ax = [1×5+2×6, 3×5+4×6] = [17,39].

3. Example Real World Applications

Computer graphics transformations, machine learning, cryptography, network models, robotics, electrical circuits and solving systems of equations.

4. Real-world problem and solution approach

Problem: A graphics program needs to rotate and scale a set of 2-D coordinates representing points on an object.

Solution approach: Represent the object points as coordinate vectors. Build scaling and rotation matrices, combine them, and use matrix multiplication to transform every point consistently.

5. Solution Code

Program file: code/Task_05_matrix_algebra.py

```
import numpy as np

# Four corners of a square: each row is [x, y]
points = np.array([
    [0.0, 0.0],
    [1.0, 0.0],
    [1.0, 1.0],
    [0.0, 1.0]
])

angle = np.deg2rad(45)

rotation = np.array([
    [np.cos(angle), -np.sin(angle)],
    [np.sin(angle),  np.cos(angle)]
])

scale = np.array([
    [2.0, 0.0],
    [0.0, 1.5]
])

# Combined transformation
transformation = rotation @ scale
```

```
# Row-vector form requires multiplication by transpose
transformed_points = points @ transformation.T

print("Original points:\n", points)
print("Transformation matrix:\n", transformation)
print("Transformed points:\n", transformed_points)
print("Determinant:", np.linalg.det(transformation))
```

E. Solution of Systems of Linear Equations

1. Mathematical Concept

Many problems can be written as Ax=b, where A contains coefficients, x contains unknowns and b contains known results. Solving the system means finding x so that multiplying A by x reproduces b.

2. Example

2x+3y=8 and 4x+y=6 becomes A=[[2,3],[4,1]], b=[8,6]. The solution is x=1, y=2 because 2(1)+3(2)=8 and 4(1)+2=6.

3. Example Real World Applications

Electrical circuit analysis, resource allocation, traffic-flow models, balancing chemical equations, computer graphics and estimating unknown parameters.

4. Real-world problem and solution approach

Problem: Two server types consume different amounts of CPU and memory. Given total CPU and memory usage, determine how many servers of each type are active.

Solution approach: Form two simultaneous equations from the resource totals and solve Ax=b using np.linalg.solve(). Verify with A @ x.

5. Solution Code

Program file: code/Task_06_linear_systems.py

```
import numpy as np

# x = number of Type A servers
# y = number of Type B servers
#
# Type A uses 4 CPU units and 8 GB memory
# Type B uses 2 CPU units and 12 GB memory
# Total observed usage: 20 CPU units and 72 GB memory

A = np.array([
    [4.0,  2.0],
    [8.0, 12.0]
])

b = np.array([20.0, 72.0])
```

```
solution = np.linalg.solve(A, b)
residual = b - A @ solution

print(f"Type A servers: {solution[0]:.0f}")
print(f"Type B servers: {solution[1]:.0f}")
print("Verification A @ solution:", A @ solution)
print("Residual:", residual)
print("Residual norm:", np.linalg.norm(residual))
```

F. Gaussian Elimination and LU Decomposition

1. Mathematical Concept

Gaussian elimination transforms Ax=b into an equivalent upper-triangular system by eliminating variables one row at a time, then solves by back-substitution. LU decomposition writes A = LU (or PA=LU with pivoting), where L is lower triangular and U is upper triangular. Once A is factored, several different b vectors can be solved efficiently.

2. Example

For three simultaneous equations, elimination removes x from rows 2 and 3, then removes y from row 3. The last equation gives z; back-substitution gives y and x.

3. Example Real World Applications

Engineering simulations, repeated linear-system solutions, finite-element methods, economic models and scientific models with many equations.

4. Real-world problem and solution approach

Problem: A simulation uses the same coefficient matrix A every second but receives a new measurement vector b.

Solution approach: Factor A once using LU decomposition. Reuse the factors for each new b instead of repeating full elimination, reducing computation.

5. Solution Code

Program file: code/Task_07_gaussian_lu.py

```
import numpy as np
from scipy.linalg import lu_factor, lu_solve

# The same coefficient matrix is reused for several measurement vectors.
A = np.array([
    [4.0, -1.0,  0.0],
    [-1.0, 4.0, -1.0],
    [0.0, -1.0,  3.0]
])

measurement_sets = [
    np.array([15.0, 10.0, 10.0]),
    np.array([16.0, 11.0,  9.0]),
    np.array([14.0, 12.0, 11.0])
]

# Factor A only once
lu, piv = lu_factor(A)

for i, b in enumerate(measurement_sets, start=1):
    x = lu_solve((lu, piv), b)
    residual = b - A @ x
```

```
    print(f"Measurement set {i}")
    print("Solution:", x)
    print("Residual norm:", np.linalg.norm(residual))
    print()

# Direct solution for comparison
print("Direct solution for first set:",
      np.linalg.solve(A, measurement_sets[0]))
```

G. Iterative Methods for Linear Systems

1. Mathematical Concept

Jacobi and Gauss-Seidel methods start with an initial guess and repeatedly improve it. For each equation, solve for one unknown using current estimates of the others. The process stops when the change between successive estimates is below a chosen tolerance.

2. Example

If successive estimates of x are 1.0, 1.8, 1.97, 1.999 and 2.000, the values are converging toward 2.

3. Example Real World Applications

Large sparse systems from heat transfer, electrical networks, structural models, image reconstruction and scientific simulations where direct matrix factorization may be expensive.

4. Real-world problem and solution approach

Problem: A grid model of temperatures produces thousands of linear equations, most of whose coefficients are zero.

Solution approach: Use an iterative method such as Jacobi or Gauss-Seidel. Track the error each iteration and stop once it falls below the tolerance.

5. Solution Code

Program file: code/Task_08_iterative_linear.py

```
import numpy as np
import matplotlib.pyplot as plt

# Simplified steady-state temperature model for five interior grid points.
# Diagonal dominance makes Jacobi and Gauss-Seidel converge.
A = np.array([
    [ 2., -1.,  0.,  0.,  0.],
    [-1.,  2., -1.,  0.,  0.],
    [ 0., -1.,  2., -1.,  0.],
    [ 0.,  0., -1.,  2., -1.],
    [ 0.,  0.,  0., -1.,  2.]
])

# Boundary influence: left boundary = 100 C, right boundary = 20 C
b = np.array([100., 0., 0., 0., 20.])
```

```
def jacobi(A, b, tol=1e-8, max_iter=500):
    D = np.diag(A)
    R = A - np.diagflat(D)
    x = np.zeros_like(b)
    errors = []

    for _ in range(max_iter):
        x_new = (b - R @ x) / D
        error = np.linalg.norm(x_new - x, ord=np.inf)
        errors.append(error)
        if error < tol:
            return x_new, errors
        x = x_new

    return x, errors

def gauss_seidel(A, b, tol=1e-8, max_iter=500):
    x = np.zeros_like(b)
    errors = []

    for _ in range(max_iter):
        old = x.copy()
        for i in range(len(b)):
            x[i] = (
                b[i]
                - A[i, :i] @ x[:i]
                - A[i, i+1:] @ old[i+1:]
            ) / A[i, i]

        error = np.linalg.norm(x - old, ord=np.inf)
        errors.append(error)
        if error < tol:
            return x, errors

    return x, errors

x_j, err_j = jacobi(A, b)
x_gs, err_gs = gauss_seidel(A, b)
x_exact = np.linalg.solve(A, b)

print("Jacobi temperatures:", x_j)
print("Gauss-Seidel temperatures:", x_gs)
print("Direct solution:", x_exact)

plt.semilogy(err_j, label="Jacobi")
```

```
plt.semilogy(err_gs, label="Gauss-Seidel")
plt.xlabel("Iteration")
plt.ylabel("Maximum change")
plt.title("Convergence of iterative temperature solution")
plt.legend()
plt.grid(True)
plt.show()
```

H. Eigenvalues and Eigenvectors

1. Mathematical Concept

For a square matrix A, an eigenvector v is a non-zero vector whose direction is unchanged by the transformation A. It satisfies Av = λv, where λ is the eigenvalue. The eigenvalue tells how much the vector is stretched, compressed or reversed.

2. Example

If Av = 3v, then v is an eigenvector and 3 is its eigenvalue. Applying A triples the magnitude of v without changing its direction.

3. Example Real World Applications

Google PageRank concepts, vibration analysis, stability of dynamical systems, principal component analysis, image compression, quantum models and network centrality.

4. Real-world problem and solution approach

Problem: A small computer network needs to identify which nodes are structurally most influential from their pattern of connections.

Solution approach: Represent the network as an adjacency matrix, compute its dominant eigenvalue and eigenvector, and normalize the dominant eigenvector to obtain an eigenvector-based node-importance measure.

5. Solution Code

Program file: code/Task_09_eigenvalues.py

```
import numpy as np

# Adjacency matrix for a small undirected network.
# Node order: A, B, C, D
A = np.array([
    [0., 1., 1., 0.],
    [1., 0., 1., 1.],
    [1., 1., 0., 1.],
    [0., 1., 1., 0.]
])

eigenvalues, eigenvectors = np.linalg.eig(A)

# Dominant eigenvalue/eigenvector
```

```
index = np.argmax(np.real(eigenvalues))
dominant_value = np.real(eigenvalues[index])
dominant_vector = np.abs(np.real(eigenvectors[:, index]))
centrality = dominant_vector / dominant_vector.sum()

nodes = ["A", "B", "C", "D"]

print("Dominant eigenvalue:", dominant_value)
print("Eigenvector-based importance:")
for node, score in zip(nodes, centrality):
    print(f"{node}: {score:.4f}")

# Verify Av = lambda*v
v = np.real(eigenvectors[:, index])
print("Verification error:",
      np.linalg.norm(A @ v - dominant_value * v))
```

I. Root Finding: Bisection and Fixed-Point Methods

1. Mathematical Concept

A root of f(x) is a value r for which f(r)=0. Bisection starts with an interval [a,b] where f(a) and f(b) have opposite signs, repeatedly halves the interval and keeps the half containing a root. Fixed-point iteration rewrites the equation as x=g(x) and repeatedly calculates $x_{n+1}=g(x_n)$.

2. Example

For $f(x)=x^3-x-2$, f(1)=-2 and f(2)=4, so a root lies between 1 and 2. Bisection repeatedly halves this interval until the root is approximated closely.

3. Example Real World Applications

Finding break-even points, solving engineering equations, computing interest rates, calibration, physical equilibrium and threshold values.

4. Real-world problem and solution approach

Problem: Determine the temperature at which a nonlinear heat-balance equation becomes zero.

Solution approach: Define the heat-balance function f(T), bracket a sign change and apply the bisection method. Stop when the interval or function value is sufficiently small.

5. Solution Code

Program file: code/Task_10_bisection_fixed_point.py

```
import numpy as np
from scipy.optimize import root_scalar
import matplotlib.pyplot as plt

# Heat-balance model:
# f(T) = 0 means heat gain equals heat loss.
def heat_balance(T):
```

```
    return T - 25 - 10*np.exp(-T/20)

# Bisection: bracket a sign change
solution = root_scalar(
    heat_balance,
    bracket=[20, 40],
    method="bisect",
    xtol=1e-10
)

# Fixed-point form:
# T = 25 + 10*exp(-T/20)
def g(T):
    return 25 + 10*np.exp(-T/20)

T = 30.0
history = [T]

for _ in range(100):
    T_new = g(T)
    history.append(T_new)

    if abs(T_new - T) < 1e-10:
        break

    T = T_new

print(f"Bisection equilibrium temperature: {solution.root:.6f} C")
print(f"Fixed-point equilibrium temperature: {history[-1]:.6f} C")
print("Fixed-point iterations:", len(history) - 1)

temps = np.linspace(20, 40, 300)
plt.plot(temps, heat_balance(temps))
plt.axhline(0, linewidth=0.8)
plt.axvline(solution.root, linestyle="--")
plt.xlabel("Temperature (C)")
plt.ylabel("Heat-balance function")
plt.title("Equilibrium temperature")
plt.grid(True)
plt.show()
```

J. Root Finding: Newton-Raphson and Secant Methods

1. Mathematical Concept

Newton-Raphson improves a guess using $x_{n+1} = x_n - f(x_n)/f'(x_n)$. It uses the tangent line to move toward a root. The secant method avoids an explicit derivative and estimates the slope using two recent points. Both can converge much faster than bisection when started well.

2. Example

For $f(x)=x^2-2$ and $x_0=1.5$, Newton gives $x_1=1.5-(0.25/3)=1.4167$, then quickly approaches $\sqrt{2}\approx1.4142$.

3. Example Real World Applications

Circuit equations, optimization subproblems, financial rate calculations, nonlinear models, calibration and solving physical equations.

4. Real-world problem and solution approach

Problem: A financial application must compute an interest rate that makes net present value equal to zero.

Solution approach: Define NPV(r) as f(r). Use Newton-Raphson when a derivative is available or the secant method when only function values are available.

5. Solution Code

Program file: code/Task_11_newton_secant.py

```
from scipy.optimize import root_scalar

# Initial investment followed by annual cash inflows
cash_flows = [-10000, 3000, 3500, 4000, 2500]

def npv(rate):
    return sum(
        cf / (1 + rate)**t
        for t, cf in enumerate(cash_flows)
    )

def d_npv(rate):
    return sum(
        -t * cf / (1 + rate)**(t + 1)
        for t, cf in enumerate(cash_flows)
        if t > 0
    )

newton = root_scalar(
    npv,
    fprime=d_npv,
    x0=0.10,
    method="newton",
    xtol=1e-12
```

```
)

secant = root_scalar(
    npv,
    x0=0.05,
    x1=0.20,
    method="secant",
    xtol=1e-12
)

print(f"IRR using Newton-Raphson: {newton.root:.6%}")
print(f"IRR using Secant method:  {secant.root:.6%}")
print(f"NPV at Newton solution:   {npv(newton.root):.6f}")
```

K. Polynomial and Nonlinear Equation Solving

1. Mathematical Concept

A polynomial equation has the form $a\square x^n + ... + a_1x + a_0 = 0$. A nonlinear system contains equations that are not all linear in the unknowns. Numerical solvers search for values that make every equation or residual close to zero.

2. Example

$x^2-5x+6=0$ has roots x=2 and x=3 because (x-2)(x-3)=0. For more complicated equations, numerical solvers estimate the roots.

3. Example Real World Applications

Kinematics, chemical equilibrium, geometric intersection, control systems, calibration and finding operating points in nonlinear models.

4. Real-world problem and solution approach

Problem: Find the intersection points between a circle and a parabola, producing two nonlinear equations in x and y.

Solution approach: Define the circle and parabola as two residual equations and use scipy.optimize.root from more than one initial guess to locate the intersections. Verify that the residuals are close to zero.

5. Solution Code

Program file: code/Task_12_nonlinear_equations.py

```
import numpy as np
from scipy.optimize import root

# Find intersections of:
# Circle: x^2 + y^2 = 4
# Parabola: y = x^2 - 1

def equations(z):
    x, y = z
```

```
    return [
        x**2 + y**2 - 4,
        y - (x**2 - 1)
    ]

for guess in ([1.0, 1.0], [-1.0, 1.0]):
    solution = root(equations, guess)

    print("Initial guess:", guess)
    print("Intersection:", solution.x)
    print("Residuals:", equations(solution.x))
    print("Converged:", solution.success)
    print()
```

L. Interpolation

1. Mathematical Concept
Interpolation estimates values between known data points. Linear interpolation draws a straight line between neighboring points. Polynomial interpolation uses a polynomial through several points. Cubic splines join smooth cubic curves while preserving smoothness between intervals.

2. Example
If temperature is 20°C at 10:00 and 24°C at 12:00, linear interpolation estimates 22°C at 11:00.

3. Example Real World Applications
Sensor calibration, missing measurement estimation, animation, geographic data, lookup tables, engineering tables and resampling signals.

4. Real-world problem and solution approach
Problem: A sensor records temperature every hour, but an application needs estimates every 10 minutes.

Solution approach: Use scipy.interpolate to build an interpolation function from the hourly measurements and evaluate it at the desired intermediate times.

5. Solution Code
Program file: code/Task_13_interpolation.py

```
import numpy as np
from scipy.interpolate import interp1d, CubicSpline
import matplotlib.pyplot as plt

# Hourly temperature measurements
hours = np.array([8, 9, 10, 11, 12, 13, 14], dtype=float)
temperature = np.array([18.0, 20.0, 22.5, 24.0, 25.0, 25.5, 25.0])

linear = interp1d(hours, temperature, kind="linear")
cubic = CubicSpline(hours, temperature)
```

```
# Estimate every 10 minutes = 1/6 hour
ten_minute_times = np.arange(8, 14 + 1/6, 1/6)

linear_estimates = linear(ten_minute_times)
cubic_estimates = cubic(ten_minute_times)

target_time = 10.5  # 10:30
print(f"Linear estimate at 10:30: {float(linear(target_time)):.2f} C")
print(f"Cubic estimate at 10:30:  {float(cubic(target_time)):.2f} C")

plt.scatter(hours, temperature, label="Hourly measurements")
plt.plot(ten_minute_times, linear_estimates, label="Linear")
plt.plot(ten_minute_times, cubic_estimates, label="Cubic spline")
plt.xlabel("Hour of day")
plt.ylabel("Temperature (C)")
plt.title("Temperature interpolation")
plt.legend()
plt.grid(True)
plt.show()
```

M. Curve Fitting and Approximation

1. Mathematical Concept

Curve fitting chooses model parameters so a mathematical function follows observed data as closely as possible. In least squares, parameters minimize $\Sigma(y_i-\hat{y}_i)^2$, the sum of squared residuals between observations and model predictions.

2. Example

If data approximately follows y=a+bx, least squares chooses a and b that minimize the squared vertical differences between measured points and the fitted line.

3. Example Real World Applications

Trend estimation, calibration, performance modelling, growth curves, sensor models, demand forecasting and experimental science.

4. Real-world problem and solution approach

Problem: Measure server response time at different user loads and estimate a mathematical relationship between load and response time.

Solution approach: Choose a plausible model, fit its parameters using scipy.optimize.curve_fit or NumPy least squares, plot observed and fitted values, and inspect residuals.

5. Solution Code

Program file: code/Task_14_curve_fitting.py

```
import numpy as np
from scipy.optimize import curve_fit
```

```
import matplotlib.pyplot as plt

# Concurrent users and measured server response time (milliseconds)
users = np.array([50, 100, 150, 200, 250, 300, 350], dtype=float)
response_ms = np.array([82, 95, 115, 145, 185, 238, 305], dtype=float)

# A simple quadratic model for increasing delay
def model(x, a, b, c):
    return a*x**2 + b*x + c

parameters, covariance = curve_fit(model, users, response_ms)
a, b, c = parameters

predicted = model(users, a, b, c)
rmse = np.sqrt(np.mean((response_ms - predicted)**2))

print(f"Fitted model: y = {a:.6f}x^2 + {b:.4f}x + {c:.2f}")
print(f"RMSE: {rmse:.2f} ms")
print(f"Predicted response at 400 users: {model(400, a, b, c):.2f} ms")

x_plot = np.linspace(users.min(), 400, 300)
plt.scatter(users, response_ms, label="Measurements")
plt.plot(x_plot, model(x_plot, a, b, c), label="Fitted curve")
plt.xlabel("Concurrent users")
plt.ylabel("Response time (ms)")
plt.title("Server response-time curve fitting")
plt.legend()
plt.grid(True)
plt.show()
```

N. Numerical Differentiation

1. Mathematical Concept

A derivative measures how quickly a quantity changes. When a formula derivative is unavailable, approximate it from nearby values. Forward difference: $f'(x) \approx [f(x+h)-f(x)]/h$. Central difference: $f'(x) \approx [f(x+h)-f(x-h)]/(2h)$, which is often more accurate for the same h.

2. Example

If distance changes from 100 m to 112 m in 2 s, average rate of change is (112-100)/2=6 m/s. Numerical differentiation applies the same idea using very small intervals.

3. Example Real World Applications

Estimating speed from position data, acceleration from velocity, gradients in optimization, rates of change in sensors and sensitivity analysis.

4. Real-world problem and solution approach

Problem: A GPS device records position at closely spaced times. Estimate instantaneous vehicle speed.

Solution approach: Apply central differences to neighboring position measurements and divide by the time interval. Compare different step sizes to understand error.

5. Solution Code

Program file: code/Task_15_numerical_differentiation.py

```
import numpy as np
import matplotlib.pyplot as plt

# GPS samples: time in seconds and position in metres
time = np.array([0, 1, 2, 3, 4, 5, 6], dtype=float)
position = np.array([0.0, 4.8, 10.2, 16.1, 22.7, 30.0, 38.0])

speed = np.zeros_like(position)

# Forward difference at the beginning
speed[0] = (position[1] - position[0]) / (time[1] - time[0])

# Central differences for interior points
for i in range(1, len(time) - 1):
    speed[i] = (
        position[i + 1] - position[i - 1]
    ) / (
        time[i + 1] - time[i - 1]
    )

# Backward difference at the end
speed[-1] = (
    position[-1] - position[-2]
) / (
    time[-1] - time[-2]
)

for t, x, v in zip(time, position, speed):
    print(f"t={t:.0f}s, position={x:.1f}m, estimated speed={v:.2f}m/s")

plt.plot(time, speed, marker="o")
plt.xlabel("Time (s)")
plt.ylabel("Estimated speed (m/s)")
plt.title("Speed estimated from GPS position data")
plt.grid(True)
plt.show()
```

O. Numerical Integration

1. Mathematical Concept
An integral represents accumulated quantity or area under a curve. The trapezoidal rule approximates the area with trapezoids. Simpson's rule approximates the curve with parabolic segments and is often more accurate for smooth functions. SciPy can use adaptive quadrature to refine the calculation automatically.

2. Example
If power is constant at 2 kW for 3 hours, energy is $\int P\,dt = 2\times3 = 6$ kWh. For changing power, numerical integration adds many small contributions over time.

3.3. Example Real World Applications
Energy consumption, distance from velocity data, probability calculations, signal energy, total flow, area and accumulated quantities.

4. Real-world problem and solution approach
Problem: A smart meter records power at regular intervals. Estimate total electrical energy consumed during the day.

Solution approach: Integrate the power-versus-time measurements numerically using trapezoidal or Simpson's rule.

5. Solution Code
Program file: code/Task_16_numerical_integration.py

```
import numpy as np
from scipy.integrate import simpson

# Smart-meter readings taken every two hours
time_hours = np.array([0, 2, 4, 6, 8, 10, 12], dtype=float)
power_kw = np.array([0.8, 1.0, 1.4, 1.8, 1.6, 1.2, 0.9])

energy_trapezoid = np.trapezoid(power_kw, time_hours)
energy_simpson = simpson(power_kw, x=time_hours)

print(f"Energy using trapezoidal rule: {energy_trapezoid:.3f} kWh")
print(f"Energy using Simpson's rule:   {energy_simpson:.3f} kWh")
print(f"Difference between estimates:  {abs(energy_simpson-energy_trapezoid):.3f}
kWh")
```

P. Ordinary Differential Equations – Euler Method

1. Mathematical Concept

An ODE describes how a quantity changes, for example dy/dt=f(t,y). Euler's method starts from a known value y_0 and steps forward: $y_{n+1} = y_n + h\, f(t_n, y_n)$. Here h is the step size. A smaller h usually improves accuracy but requires more computation.

2. Example

For dy/dt=y, y(0)=1 and h=0.1, Euler gives $y_1=1+0.1(1)=1.1$, then $y_2=1.1+0.1(1.1)=1.21$.

3. Example Real World Applications

Population growth, cooling, charging capacitors, motion, epidemic models and other systems that evolve over time.

4. Real-world problem and solution approach

Problem: Predict cooling of a hot object using Newton's law of cooling.

Solution approach: Express the cooling law as an ODE, choose an initial temperature and time step, then repeatedly apply Euler's update equation and plot temperature versus time.

5. Solution Code

Program file: code/Task_17_euler_ode.py

```
import numpy as np
import matplotlib.pyplot as plt

# Newton's law of cooling:
# dT/dt = -k(T - T_ambient)
T_ambient = 25.0
T_initial = 90.0
k = 0.12
h = 1.0  # minutes
end_time = 30.0

def cooling_rate(T):
    return -k * (T - T_ambient)

time = np.arange(0, end_time + h, h)
temperature = np.zeros_like(time)
temperature[0] = T_initial

for i in range(len(time) - 1):
    temperature[i + 1] = (
        temperature[i] + h * cooling_rate(temperature[i])
    )

# Exact solution for validation
exact = T_ambient + (T_initial - T_ambient) * np.exp(-k * time)
```

```
print(f"Euler temperature after 30 min: {temperature[-1]:.2f} C")
print(f"Exact temperature after 30 min: {exact[-1]:.2f} C")
print(f"Final absolute error: {abs(temperature[-1]-exact[-1]):.4f} C")

plt.plot(time, temperature, marker="o", label="Euler")
plt.plot(time, exact, label="Exact solution")
plt.xlabel("Time (minutes)")
plt.ylabel("Temperature (C)")
plt.title("Cooling of a hot object")
plt.legend()
plt.grid(True)
plt.show()
```

Q. Numerical Solution of ODEs Using SciPy

1. Mathematical Concept

More advanced ODE solvers use higher-order methods and automatically adjust step sizes. scipy.integrate.solve_ivp numerically solves dy/dt=f(t,y) over a time interval and returns approximated values at requested times.

2. Example

For exponential decay dy/dt=-ky, solve_ivp computes y(t) from y(0) without manually performing every update.

3.3. Example Real World Applications

Control systems, epidemiology, robotics, chemical reactions, electrical circuits, mechanical systems and dynamic simulations.

4. Real-world problem and solution approach

Problem: Model the voltage in an RC circuit as it charges or discharges over time.

Solution approach: Write the circuit differential equation as a Python function and pass it to solve_ivp with initial voltage and time interval. Plot the numerical solution.

5. Solution Code

Program file: code/Task_18_scipy_ode.py

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

# RC charging circuit:
# dV/dt = (Vs - V)/(R*C)
Vs = 5.0          # supply voltage (V)
R = 1000.0        # resistance (ohms)
```

```
C = 0.001         # capacitance (farads)
V0 = 0.0          # initial capacitor voltage

def rc_circuit(t, V):
    return [(Vs - V[0]) / (R * C)]

t_eval = np.linspace(0, 5, 200)

solution = solve_ivp(
    rc_circuit,
    (0, 5),
    [V0],
    t_eval=t_eval,
    rtol=1e-9,
    atol=1e-11
)

# Exact RC charging curve for validation
exact = Vs * (1 - np.exp(-solution.t / (R * C)))

print("Solver success:", solution.success)
print(f"Capacitor voltage at 5 s: {solution.y[0, -1]:.4f} V")
print(f"Maximum error against exact solution:
{np.max(np.abs(solution.y[0]-exact)):.3e} V")

plt.plot(solution.t, solution.y[0], label="solve_ivp")
plt.plot(solution.t, exact, linestyle="--", label="Exact")
plt.xlabel("Time (s)")
plt.ylabel("Capacitor voltage (V)")
plt.title("RC circuit charging")
plt.legend()
plt.grid(True)
plt.show()
```

R. Numerical Optimization

1. Mathematical Concept
Optimization finds input values that minimize or maximize an objective function. If minimizing f(x), an optimum x* has an objective value no larger than nearby feasible values. Some problems also include constraints such as x≥0 or resource limits.

2. Example
For $f(x)=(x-3)^2+2$, the minimum occurs at x=3 because the squared term becomes zero, giving minimum value 2.

3.3. Example Real World Applications
Cloud resource allocation, network routing, machine-learning training, scheduling, engineering design, cost reduction and parameter tuning.

4. Real-world problem and solution approach
Problem: A cloud service wants to choose the allocation of two server types that minimizes operating cost while meeting a minimum processing-capacity requirement.

Solution approach: Define operating cost as the objective function, express the minimum capacity as an inequality constraint, add non-negative bounds, and solve using scipy.optimize.minimize. If whole servers are required, compare nearby integer combinations.

5. Solution Code
Program file: code/Task_19_optimization.py

```
import numpy as np
from scipy.optimize import minimize

# x = number/capacity units of Type A server
# y = number/capacity units of Type B server
#
# Cost per unit: A = 12, B = 18
# Capacity per unit: A = 100, B = 170
# Required total capacity: at least 1000

def cost(z):
    x, y = z
    return 12*x + 18*y

def capacity_constraint(z):
    x, y = z
    return 100*x + 170*y - 1000

constraints = {
    "type": "ineq",
    "fun": capacity_constraint
}
```

```
bounds = [(0, None), (0, None)]

solution = minimize(
    cost,
    x0=[5, 3],
    method="SLSQP",
    bounds=bounds,
    constraints=constraints
)

x, y = solution.x

print("Optimization success:", solution.success)
print(f"Type A allocation: {x:.3f}")
print(f"Type B allocation: {y:.3f}")
print(f"Total capacity: {100*x + 170*y:.2f}")
print(f"Minimum planning cost: {solution.fun:.2f}")

# If whole servers are required, test nearby integer combinations.
best_integer = None

for a in range(0, 20):
    for b in range(0, 20):
        capacity = 100*a + 170*b
        if capacity >= 1000:
            total_cost = 12*a + 18*b
            if best_integer is None or total_cost < best_integer[0]:
                best_integer = (total_cost, a, b, capacity)

print("Best nearby integer solution:", best_integer)
```

5. Random Numbers and Monte Carlo Methods

1. Mathematical Concept

Monte Carlo methods estimate quantities by repeated random sampling. If an event occurs with probability p, then after N simulations its observed fraction approaches p as N becomes large. A classic Example estimates π by randomly sampling points in a square and counting how many fall inside a quarter circle: $\pi \approx 4 \times (\text{inside/total})$.

2. Example

If 7,850 of 10,000 random points fall inside the quarter circle, $\pi \approx 4 \times 0.785 = 3.14$.

3. Example Real World Applications

Risk analysis, reliability, queue simulation, finance, uncertainty analysis, computer graphics, physical simulations and estimating difficult integrals.

4. Real-world problem and solution approach

Problem: Estimate the probability that a network server becomes overloaded when incoming demand is uncertain.

Solution approach: Generate many random demand scenarios, evaluate whether each exceeds capacity, and estimate overload probability as overloaded simulations divided by total simulations.

5. Solution Code

Program file: code/Task_20_monte_carlo.py

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

capacity = 1000.0
mean_demand = 850.0
std_demand = 120.0
N = 100_000

# Random demand scenarios
demand = rng.normal(mean_demand, std_demand, N)

overloaded = demand > capacity
overload_probability = overloaded.mean()

print(f"Estimated overload probability: {overload_probability:.4%}")
print(f"Number of overload cases: {overloaded.sum()} out of {N}")

# Running probability to show convergence
running_probability = np.cumsum(overloaded) / np.arange(1, N + 1)

indices = np.unique(
```

```
    np.logspace(1, np.log10(N), 200).astype(int)
) - 1

plt.semilogx(
    indices + 1,
    running_probability[indices]
)
plt.xlabel("Number of simulations")
plt.ylabel("Estimated overload probability")
plt.title("Monte Carlo estimate of server-overload risk")
plt.grid(True)
plt.show()
```